

Kaggle Competition Mastery — End-to-End ML Pipeline

A step-by-step pipeline from raw data to final submission, in the order you'd actually run it. Defaults lean tabular/structured data (the majority of Kaggle competitions); Phase 8 branches out for vision, NLP, and time series.

Tags: `[Core]` = do this every competition · `[High-Impact]` = usually the biggest score movers · `[Advanced]` = final-mile, marginal-gains territory

The Order, At a Glance

0. Frame the competition
 1. Environment & tooling
 2. Load & first-look at data
 3. Exploratory data analysis
 4. Clean the data
 5. Validation strategy
 6. Feature engineering
 7. Fast baselines
 8. Model selection & training
 9. Hyperparameter optimization
 10. Evaluate & diagnose
 11. Ensembling
 12. Score-boosting extras
 13. Submit & iterate
-

Phase 0 — Frame the Competition `[Core]`

Goal: understand what's actually being scored before writing any code.

- Read the evaluation metric twice. AUC, LogLoss, RMSE, RMSLE, MAP@K, Quadratic Weighted Kappa, F-beta all reward different things — optimizing the wrong metric locally is the #1 avoidable mistake.
- Check the train/test split logic: random, time-based, or grouped (e.g. same user/store can't appear in both)? This decides your CV scheme in Phase 5.
- Read the Data tab + pinned Discussion posts before touching code — leakage warnings, dataset quirks, and target definitions are usually posted there.
- Skim 1-2 public notebooks for structure, not to copy.
- Fix a random-seed convention (numpy, framework, model) and reuse it everywhere for comparable runs.

Tools: Kaggle Discussions/Code tabs, `kaggle competitions download -c <name>`

Phase 1 — Environment & Tooling ([core])

Goal: set up once so every later phase is fast.

- Compute: Kaggle Notebooks (free T4/P100/TPU) or Colab for GPU work; local for cheap CPU iteration.
- GPU-accelerate the tabular stack from day one — this is what current top workflows actually run on:
 - `cudf.pandas` — zero-code-change GPU acceleration for existing pandas code (`%load_ext cudf.pandas`)
 - `cudf-polars` — GPU engine for Polars
 - GPU builds of **LightGBM**, **XGBoost**, **CatBoost** (`device='gpu'`) / (`task_type='GPU'`)
 - **cuML** — GPU drop-in for scikit-learn estimators
- Experiment tracking from the first run: **Weights & Biases**, **MLflow**, or a plain CSV/sheet log (params, CV score, LB score, notes) — anything beats memory.
- An LLM coding agent (e.g. Claude Code) to help generate/test feature-engineering variants and boilerplate faster is an increasingly normal part of the efficiency stack — use it to widen how many ideas you can test, not to skip understanding the data.
- `conda`/`venv` with pinned versions for reproducibility.

Tools: Kaggle Notebooks, Colab, RAPIDS (`cudf`, `cuml`), LightGBM/XGBoost/CatBoost (GPU builds), Weights & Biases, MLflow

Phase 2 — Load & First-Look ([core])

Goal: get data in memory efficiently and sanity-check shape before analysis.

- Load with `pandas`/`cudf` (small-medium), `polars` (fast, memory-efficient), or `dask`/`Vaex` (larger-than-RAM).
- Downcast dtypes immediately (`float64→float32`, `int64→int32`), low-cardinality strings → `category`) — typically cuts memory 50-70%.
- Checklist: `.shape`, `.info()`, `.describe()`, `.head()`, target distribution, duplicate rows, train/test column mismatch.

Charts: target distribution (histogram/countplot), missing-value bar chart

Tools: pandas, polars, cuDF, Dask, Vaex, pyarrow

Phase 3 — Exploratory Data Analysis ([Core / High-Impact])

Goal: find the patterns Phase 6 will exploit. This is the step top competitors spend the most disciplined time on, not a box to check.

- Automated first pass: **Sweetviz**, **AutoViz**, **D-Tale** for a fast full-dataset overview — **ydata-profiling** is a solid alternative report style.

- Missingness: `missingno` matrix + bar chart — look for *patterns* in what's missing, often informative rather than random.
- Univariate: histogram/KDE per numeric feature, countplot per categorical.
- Bivariate vs. target: boxplot/violin (categorical vs. numeric target), scatter/hexbin (numeric vs. numeric), target-rate bar chart (categorical vs. classification target).
- Correlation: Pearson/Spearman heatmap, clustermap to group correlated blocks.
- Outliers: boxplot, IQR-bounded scatter.
- Time series: line plot over time, seasonal decomposition, ACF/PACF plots.
- **Adversarial validation** `[High-Impact]`: train a classifier to distinguish train rows from test rows. A high AUC means your train/test distributions differ — your CV won't match the leaderboard, and it's better to know that now than after a leaderboard shake-up.

Charts: histogram, KDE, boxplot, violin, scatter/hexbin, correlation heatmap, clustermap, missingno matrix, ACF/PACF, seasonal decomposition

Tools: Sweetviz, AutoViz, D-Tale, ydata-profiling, missingno, matplotlib, seaborn, plotly

Phase 4 — Clean the Data `[Core]`

Goal: fix what would otherwise quietly cap your ceiling.

- Missing values: tree models often do better with raw `NaN` (LightGBM/XGBoost/CatBoost handle it natively) or an `is_missing` flag column than with imputation, which can destroy signal. Where you do impute: `SimpleImputer`, `KNNImputer`, `IterativeImputer` (MICE).
- Outliers: cap/winsorize, or log/Box-Cox/Yeo-Johnson transform skewed numerics — understand *why* a value is extreme before dropping it.
- Inconsistent categories (typos, casing, "USA" vs "United States"): fuzzy-match with `rapidfuzz`/`thefuzz`.
- Dedup, fix dtypes, align train/test columns.

Tools: scikit-learn (`SimpleImputer`, `KNNImputer`, `IterativeImputer`), scipy.stats, rapidfuzz

Phase 5 — Validation Strategy `[Core – get this wrong and nothing downstream is trustworthy]`

Goal: build a CV scheme that actually predicts your leaderboard score.

- Match the split to how train/test were actually divided (Phase 0):
 - Random → `KFold`/`StratifiedKFold` (classification, especially imbalanced)
 - Grouped entities (user/store/patient shouldn't leak across folds) → `GroupKFold`/`StratifiedGroupKFold`
 - Time-based → `TimeSeriesSplit`, or a manual rolling window matching the competition's real time cutoff

- 5-fold is a reasonable default; go higher for small/noisy data, or repeated K-Fold if you have the compute budget.
- Track fold-to-fold variance (boxplot of per-fold scores) — a wide spread means the CV number isn't trustworthy yet.
- Once validated, trust your CV over the public leaderboard — the public LB is a small, noisy slice of the test set.

Charts: boxplot of per-fold scores

Tools: scikit-learn (`model_selection`) (KFold, StratifiedKFold, GroupKFold, TimeSeriesSplit)

Phase 6 — Feature Engineering ([High-Impact – usually the single biggest lever on tabular data])

Goal: turn what EDA found into columns the model can use.

- Numerical: binning, log/sqrt/Box-Cox, ratios and differences between related columns, polynomial/interaction terms.
- **Groupby aggregations** — the workhorse: for each categorical column, compute (`mean/std/min/max/count/nunique/skew`) of numeric columns by group. With GPU-accelerated pandas you can systematically test thousands of (`(col1, col2, stat)`) combinations instead of hand-picking a handful.
- Categorical encoding: One-Hot (low cardinality), Target/Mean encoding (fit inside CV folds only, or it leaks), Frequency encoding, Weight-of-Evidence (binary classification) — or let CatBoost/LightGBM handle categoricals natively.
- Datetime: year/month/day/dayofweek/hour, cyclical sin/cos encoding, time-since-event, holiday flags.
- Text columns even inside a tabular comp: TF-IDF, word/char counts, basic sentiment/readability scores.
- Feature selection: permutation importance, SHAP values, **null importance** (compare real importance against importance on a shuffled target — anything that doesn't beat the shuffled baseline is noise), Boruta, (`RFE`).
- Automated discovery: (`featuretools`) (deep feature synthesis, relational data), (`tsfresh`) (time series).
- Dimensionality reduction (as features or for viz): PCA, UMAP, t-SNE.

Charts: feature-importance bar chart, SHAP summary/beeswarm plot, null-importance comparison histogram

Tools: category_encoders, feature-engine, featuretools, tsfresh, scikit-learn preprocessing, SHAP, Boruta

Phase 7 — Fast Baselines ([Core])

Goal: get a real number on the board before optimizing anything.

- Dumbest-possible baseline first (mean/median or majority-class prediction) — sanity-checks your metric and pipeline end-to-end.
- Then a default-params LightGBM, or **AutoGluon** (`TabularPredictor`) — a legitimate one-call baseline that stacks/ensembles automatically and gives a strong reference score in minutes.
- This baseline is your reference point for "did that feature/tweak actually help."

Tools: LightGBM (default params), AutoGluon

Phase 8 — Model Selection & Training [`High-Impact – branches by data type`]

Tabular/structured (most Kaggle competitions):

- Gradient-boosted trees are still the default winners: **LightGBM** (fastest, strong default), **XGBoost** (robust, huge tuning surface), **CatBoost** (best native categorical handling, less tuning needed).
- Train several — diversity across model families matters more for your eventual ensemble (Phase 11) than any single model's score.
- Neural nets on tabular data (TabNet, FT-Transformer via `pytorch-tabular`) rarely beat GBMs solo but often add useful ensemble diversity.
- **AutoGluon** throughout as a strong, low-effort reference.

Computer vision:

- Transfer learning from pretrained backbones via `timm` (PyTorch Image Models) — EfficientNet/ConvNeXt/ResNet — fine-tuned on the competition data; `fastai` for a faster path to a solid baseline.
- Fine-tuning specifics: discriminative learning rates (lower for early/frozen layers, higher for the head), progressive unfreezing, mixed-precision training.
- Augmentation: `albumentations`.

NLP:

- Fine-tune pretrained transformers via HuggingFace `transformers` (RoBERTa/DeBERTa are common strong baselines).
- Fine-tuning specifics: mixed precision (fp16/bf16), LR warmup + linear/cosine decay, layer-wise LR decay, gradient accumulation for larger effective batch size, `PEFT`/LoRA when GPU memory is tight.
- `sentence-transformers` for embedding-based approaches.

Time series:

- LightGBM with lag/rolling/window features often beats dedicated TS models on Kaggle.
- `statsmodels` (ARIMA/SARIMA), `Prophet` for classical approaches; `pytorch-forecasting` (N-BEATS, Temporal Fusion Transformer) for deep learning.

Tools: LightGBM, XGBoost, CatBoost, AutoGluon, pytorch-tabular, timm, fastai, albumentations, HuggingFace transformers, sentence-transformers, PEFT/LoRA, statsmodels, Prophet, pytorch-forecasting

Phase 9 — Hyperparameter Optimization [High-Impact]

Goal: squeeze each model before committing compute to ensembling.

- **Optuna** is the current standard — Bayesian optimization with pruning, which kills bad trials early and saves compute.
- Tune against the CV-scored pipeline, not a single train/val split — a single split overfits your tuning to that split.
- Wide search first (log-uniform ranges), then narrow around the best region.

Charts: Optuna optimization-history plot, parallel-coordinate plot, hyperparameter-importance plot

Tools: Optuna, Hyperopt, scikit-optimize

Phase 10 — Evaluate & Diagnose [Core]

Goal: know *why* a model scores what it scores before trusting or ensembling it.

- Classification: confusion matrix, ROC curve, precision-recall curve (especially for imbalanced targets), calibration curve.
- Regression: predicted-vs-actual scatter, residual plot.
- Both: learning curves (train vs. val score over boosting rounds/epochs) to catch over/underfitting.
- Feature importance: built-in gain/split, permutation importance, SHAP (summary, dependence, waterfall plots) — both for interpretability and to sanity-check the model learned something sensible rather than a leak.

Charts: confusion matrix heatmap, ROC curve, precision-recall curve, calibration curve, residual plot, predicted-vs-actual scatter, learning curve, SHAP summary/dependence/waterfall

Tools: scikit-learn.metrics, SHAP, yellowbrick, eli5

Phase 11 — Ensembling [High-Impact – this is where leaderboard positions actually separate]

Goal: combine diverse models into something better than any single one.

- **Weighted averaging/blending:** simple to start — optimize weights against out-of-fold (OOF) predictions with `scipy.optimize`.
- **Hill climbing:** greedily add whichever model (with replacement) most improves the OOF ensemble score at each step. Scales to combining hundreds of models and is behind a lot of recent winning solutions.

- **Stacking:** train a meta-model (often logistic/ridge regression, or a shallow LightGBM) on OOF predictions from your base models as features. Multi-level stacking (a stack of stacks) is standard in top solutions.
- Diversity is the actual lever: vary model family (GBM/NN/linear), feature subsets, seeds, and even target framing (predict the raw target vs. a ratio vs. a residual vs. log-target — different framings of the same problem is a known top-competitor trick).
- Check diversity directly: correlation heatmap between each model's OOF predictions — near-1.0 correlation means little ensembling benefit.

Charts: OOF prediction correlation heatmap, ensemble-weight bar chart

Tools: `scipy.optimize`, `mlxtend`, `vecstack`, manual OOF stacking

Phase 12 — Score-Boosting Extras [Advanced — final-mile gains]

- **Test-Time Augmentation (TTA)** [High-Impact, vision/NLP]: average predictions across multiple augmented versions of each test sample (flips/crops/rotations for images, back-translation for text) — close to a free score boost.
- **Pseudo-labeling** [Advanced]: predict on test data, add high-confidence predictions back as extra training data, retrain — then fine-tune a final pass on the originally-labeled data only, so your own errors don't compound.
- **Post-processing** [Advanced]: threshold tuning for F1/F-beta metrics; probability calibration (`CalibratedClassifierCV`) — Platt scaling / isotonic regression) when the metric is probability-sensitive (LogLoss).
- **Data augmentation** [Advanced, vision/NLP]: `alumentations` (image), back-translation/`nlpaug` (text), mixup/cutmix.
- **Strengthen the final model** [High-Impact, do this]: once the pipeline and hyperparameters are locked, retrain across all CV folds and average fold predictions for the final test set, rather than one single retrain-on-everything — standard practice for the last percentage points.

Tools: `scikit-learn` `CalibratedClassifierCV`, `alumentations`, `nlpaug`

Phase 13 — Submit & Iterate [Core]

- Match `sample_submission.csv` exactly — column names, row order, ID format. A large share of "why is my score 0" issues are formatting, not modeling.
- Submit via `kaggle competitions submit -c <name> -f submission.csv -m "message"` or the UI.
- Log every submission: CV score, public LB score, what changed. The CV-vs-LB gap over time tells you whether your validation is trustworthy or drifting.
- After the deadline: read winning-solution write-ups in Discussions — the highest-value 30 minutes you'll spend post-competition, and how this technique list stays current.

Tools: Kaggle API

Quick Reference — Tool Stack by Layer

Layer	Standard	GPU / Fast Path
DataFrames	pandas	<code>cudf.pandas</code> , Polars (+ <code>cudf-polars</code>)
Big data	Dask, Vaex	<code>dask-cudf</code>
EDA/profiling	Sweetviz, AutoViz, D-Tale, ydata-profiling	—
Visualization	matplotlib, seaborn	plotly (interactive)
Classical ML	scikit-learn	cuML
Tabular models	LightGBM, XGBoost, CatBoost	GPU builds of same
AutoML baseline	AutoGluon	—
Vision	timm, fastai	—
NLP	HuggingFace transformers, PEFT/LoRA	—
Time series	statsmodels, Prophet, pytorch-forecasting	—
HPO	Optuna	—
Interpretability	SHAP, eli5, yellowbrick	—
Experiment tracking	Weights & Biases, MLflow	—

Quick Reference — Metric → What It Actually Rewards

Metric	Used for	Strategy note
RMSE / MAE	Regression	RMSE punishes big misses harder — check whether outliers are skewing it
RMSLE	Regression, skewed targets	Effectively scores on log scale — log-transform the target and train on RMSE
LogLoss	Probabilistic classification	Rewards well- <i>calibrated</i> probabilities, not just the right class — calibrate
AUC	Binary classification	Rank-based — calibration doesn't matter, ranking does
F1 / F-beta	Imbalanced classification	Post-training threshold tuning often gains more than reworking the model
MAP@K	Ranking/recommendation	Order of the top-K predictions matters — optimize ranking, not raw score
Quadratic Weighted Kappa	Ordinal classification	Penalizes being "far off" more than "slightly off" — treat as ordinal, not nominal
SMAPE	Forecasting	Symmetric — under- and over-forecasting penalized similarly

Further Reading

- NVIDIA/Kaggle Grandmasters — "The Kaggle Grandmasters Playbook: 7 Battle-Tested Modeling Techniques for Tabular Data" (developer.nvidia.com)
- RAPIDS cuDF docs — github.com/rapidsai/cudf
- AutoGluon's Kaggle tutorial — auto.gluon.ai
- Every competition's Discussions tab, read after it closes